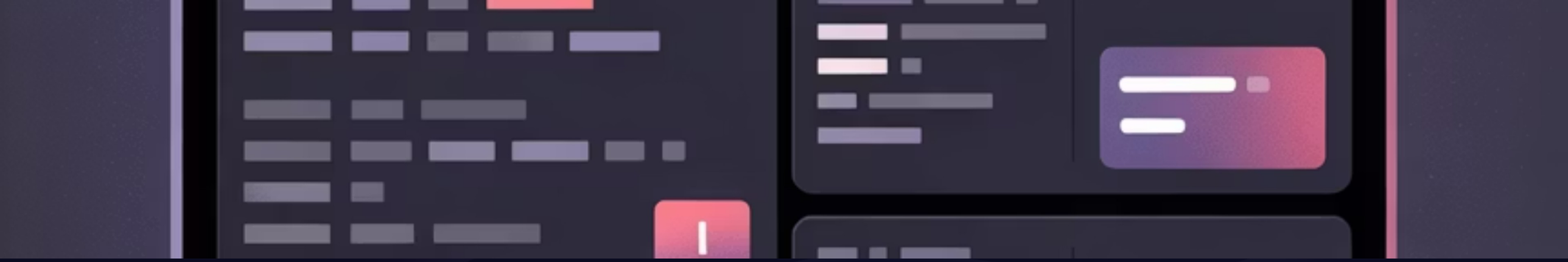
PoisonChain 🔗☠️

A malicious npm package just dropped. How many repos in your org are affected, who maintains them, and which builds ran during the attack window? **PoisonChain automatically maps the blast radius of an npm supply chain attack across your entire organization** — scanning thousands of repositories to identify affected projects, their maintainers, and compromised build pipelines, then generating per-team incident response dashboards. What would take weeks of manual analysis, done in 2 hours.

NPM SUPPLY CHAIN SECURITY INCIDENT RESPONSE CI/CD FORENSICS

GitHub: https://github.com/windshock/PoisonChain



The Attack: axios@1.14.1 Compromised

On March 30–31, 2026, attackers compromised the axios maintainer account and published two malicious versions: **axios@ 1.14.1** and **axios@ 0.30.4**. These injected **plain-crypto-js@ 4.2.1**, whose postinstall hook silently deployed a cross-platform RAT. Google GTIG attributed the attack to **TA444 / BlueNoroff** (DPRK-linked UNC1069).

⏱️ Attack Window

~3 hours

📦 Malicious Packages

axios@1.14.1, axios@0.30.4, plain-crypto-js@4.2.1

🎯 Attribution

TA444 / BlueNoroff (UNC1069) — DPRK state-sponsored

🌐 C2 Server

sfrclak.com:8000 (Hostwinds AS54290, Dallas TX)

Maintainer's Disclosure

Root Cause: How the Maintainer Account Was Compromised

The maintainer disclosure confirms the compromise involved targeted social engineering, malware on the maintainer's endpoint, and abuse of legitimate publishing rights from the compromised machine. 2FA was enabled, but the bypass method was not disclosed.

Read Full Disclosu

Targeted Social Engineering

Confirmed in the disclosure.

RAT on Endpoint

Malware was installed on the maintainer's device.

Abused Publishing Rights

Malicious releases were published from the compromised machine.

2FA Enabled

The bypass method was not disclosed.

What PoisonChain Answers for Security Teams

In the chaos of a software supply chain attack, time is critical. PoisonChain cuts through the noise, providing immediate, actionable intelligence to guide your incident response.

Affected Repositories

Identifies every repository in your organization that imported the malicious package, defining the immediate blast radius.

Project Maintainers

Pinpoints the maintainers and teams responsible for each affected project, facilitating rapid communication and delegation.

Compromised Builds

Traces CI/CD pipelines to reveal which builds executed within the attack window, providing a forensic timeline.

Code Execution Risk

Assesses whether your CI/CD pipelines actually executed the malicious code, clarifying the extent of immediate exposure.

Credential Compromise

Highlights potential credential exposure based on pipeline access, guiding urgent rotation and revocation efforts.

Developers Impacted

Lists developers who have committed to affected repositories, identifying individuals whose credentials may need rotation.

Downstream Dependencies

Maps secondary blast radius by identifying other projects or services that rely on the directly affected packages.

Why Build Logs, Not Lockfiles

The core of this attack is **evidence destruction**. The malware runs, leaves minimal traces, and then cleans up after itself.

01

1. Install runs

Dependency installation triggers the malicious `postinstall` hook.

02

2. Payload executes

The hook downloads a RAT and runs it inside the build environment.

03

3. Evidence disappears

`setup.js` is deleted and `package.json` is swapped with a harmless stub.

04

4. Workspace may look clean on casual inspection

By the time anyone checks `node_modules`, the attack has already covered its tracks.

Lockfiles show intent. Jenkins logs show execution. That difference is what turns suspicion into evidence.

Attack Chain: Technical Deep Dive

Visual roadmap of the attack sequence. Each stage card below expands on one phase of the chain.

01

Stages 1–2

Malicious Package Publication & Dropper Malicious package publication and the postinstall dropper mechanism.

02

Stage 3

Platform-Specific RAT Deployment The payload branches into macOS, Linux, and Windows execution paths.

03

Stages 4–5

C2 Communication & Cleanup The malware connects to command-and-control and begins hiding its tracks.

04

Stage 6

Credential Exposure Surface Potential credential exposure across build and development environments.

05

Stage 7

3-Hour Attack Timeline A concise end-to-end view of how the intrusion unfolded over time.

Use the stage cards that follow to trace each phase in detail. This overview is designed to orient the reader before the deeper technical breakdown.

Stage 1-2: Malicious Package Publication & Dropper

Stage 2: Dropper (setup.js)

The package's postinstall hook auto-ran `node setup.js` during dependency installation.

Obfuscation layers:

- Reversed Base64 payload text
- XOR encryption with key: `OrDeR_7077`
- Per-character offset: `7*i²%10`

Key detail: Fetched platform payload from `sfrclak.com:8000/6202033`

Main early artifact: `node_modules/plain-crypto-js/`

After execution, the dropper performed self-deletion and swapped `package.json` to reduce traces.

> This obfuscation scheme was designed to frustrate static analysis and delay detection during the critical first minutes of the attack window.

Stage 3: Platform-Specific RAT Deployment

After platform detection, the dropper delivered one of three RAT variants. Each was optimized for its OS but shared the same operator workflow

macOS

- Mach-O C++ binary, ~657KB
- Placement: /Library/Caches/com.apple.act.mond
- Beaconing: ~60-second intervals
- Capabilities: remote command execution, discovery, traversal, process control
- Attribution: overlaps with WAVESHAPER/UNC1069 tradecraft

Linux

- Python script, ~12KB
- Placement: /tmp/ld.py via nohup
- No automatic persistence; process remains active via nohup while running
- Note: peinject had a Linux porting bug, but runscript remained functional
- Ideal for: build servers and containerized environments

Windows

- Fileless PowerShell loader using .NET Reflection
- Placement: %PROGRAMDATA%\wt.exe
- Persistence: Registry Run key for auto-start
- Capabilities: in-memory loading, privilege discovery, command execution
- Advantage: avoided dropping full binary at first, reducing file artifacts

Key insight: Same operator workflow across all three platforms allowed single C2 interface for all environments.

Stage 4-5: C2 Communication & Cleanup

Stage 4: Command-and-Control Communication

Beacon Profile:

- HTTP(S)-style check-ins with host metadata
- IE8-style User-Agent to blend into legacy traffic
- ~60-second intervals for stable operator state

Command Set (shared across all platforms):

- kill: terminate the implant or selected processes
- runscript: execute supplied script or command block
- rundir: enumerate or operate within target directory
- peinject: inject or stage PE payload

Operational advantage: Same C2 workflow allowed operator to move from discovery to execution to cleanup without changing tooling.

Stage 5: Evidence Destruction and Cleanup

Cleanup was integrated into malware lifecycle, not an afterthought:

- Self-deletion: setup.js removed itself after payload handoff
- Metadata rollback: package.json reverted to v4.2.0
- Directory residue: node_modules/plain-crypto-js/ remained as main trace
- Temporary placement: payloads used cache, temp, or memory-backed locations
- Process cleanup: transient launchers detached execution from install process

Result: Reduced dwell-time footprint while preserving runtime capability to complete mission.

Credential Exposure Surface

The RAT had access to multiple credential sources within the build and development environment. These represent the potential exposure surface, not necessarily confirmed exfiltration.

- **npm tokens and registry credentials**
- **CI/CD secrets** (deployment keys, API tokens)
- **Cloud access keys** (artifact storage, logs, publishing)
- **SSH keys** on workstations or build agents
- **Service account tokens** (dashboards, monitoring, artifacts)
- **Browser/session data** on developer endpoints with active logins

Key risk: Build and install environments often expose secrets in environment variables, config files, and cached authentication material.

Stage 7: Attack Timeline & Forensic Evidence

The attack unfolded over a short publication window of roughly 3 hours, beginning with **plain-crypto-js@ 4.2.1**, then **axios@ 1.14.1** at **00:21 UTC**, and **axios@ 0.30.4** at **01:00 UTC**. The packages were unpublished around **03:15 UTC** to **03:51 UTC**, depending on the source.

- **Publication sequence:** plain-crypto-js@4.2.1 first, then axios@1.14.1 at 00:21 UTC, then axios@0.30.4 at 01:00 UTC.
- **Execution:** the postinstall hook auto-runs `setup.js`, which fetches a platform-specific RAT, deletes itself, and swaps `package.json`.
- **Persistence:** Windows establishes automatic persistence via a Registry Run key; macOS and Linux do not set automatic persistence, but active processes can remain and receive C2 commands.
- **Cleanup:** `setup.js` self-deletes, `package.json` is reverted to a clean stub, but artifacts remain on host systems.

Forensic evidence: Jenkins build logs are critical for proving what executed during the attack window. Lockfiles are reliable evidence, while `npm ls` and `npm audit` can be fooled by the package.json swap. Corroborating host artifacts include `node_modules/plain-crypto-js/`, `/tmp/ld.py`, `/Library/Caches/com.apple.act.mond`, and Windows Run-key artifacts.

> Key takeaway: use Jenkins logs plus lockfiles and host artifacts together; they provide the clearest reconstruction of execution, exposure, and persistence.

Indicators of Compromise

Cross-reference these IOCs across your environment immediately. **AV detection rates are critically low — do not rely on signature-based detection alone.**

Category	Indicator	Notes
Malicious Packages	axios@1.14.1, axios@0.30.4, plain-crypto-js@4.2.1	Safe: axios@1.14.0 (has SLSA provenance)
C2 Domain	sfrclak.com:8000	Backup: callnrwise.com (same IP, registered 53 min prior)
C2 IP	142.11.206.73	Hostwinds AS54290
Forensic Marker	node_modules/plain-crypto-js/ directory exists	Never a legitimate axios dependency — proof dropper executed

IOC Reference: Artifact Hashes & Paths

Technical reference card with detailed artifact information:

Category	Indicator	Details	Notes
macOS Artifact	Path	/Library/Caches/com.apple.act.mond	
	SHA-256	92ff08773995ebc8d55ec4b8e1a225d0d1e51efa4e-f88b8849d0071230c9645a	
Linux Artifact	Path	/tmp/ld.py	
	SHA-1	fcb81618bb15edfdedfb638b4c08a2af9cac9ec-fa551af135a8402bf980375cf	
Windows Artifact	Path	%PROGRAMDATA%\wt.exe + Registry Run key	
	Registry	HKCU\...\Run\MicrosoftUpdate → C:\ProgramData\system.bat	
Network Signature	User-Agent	mozilla/4.0 (compatible; msie 8.0; windows nt 5.1; trident/4.0)	IE8/WinXP UA on macOS/Linux is an immediate anomaly flag
Backup C2	Domain	callnrwise.com (same IP as sfrclak.com, registered 53 min prior)	
	IP	142.11.206.73 (Hostwinds AS54290)	
	Registered	2026-03-30 16:03 UTC	8 hours before first malicious release

Attribution: TA444 / BlueNoroff

Hunt.io attributed this attack to **TA444 / BlueNoroff** (DPRK-linked Lazarus subgroup) based on six independent HIGH-confidence evidence layers. Google GTIG tracks the same group as **UNC1069**. SANS links this to the broader **TeamPCP campaign** — four additional OSS projects compromised in March 2026.

Infrastructure ETag Match

C2 server shares HTTP ETag with confirmed DPRK JustJoin campaign server (23.254.167.216) — identical deployment pipeline.

NukeSped / WAVESHAPER

macOS Mach-O classified as NukeSped (Lazarus-exclusive). Elastic confirmed WAVESHAPER/UNC1069 code overlap.

SSH Key Cluster

Three coordinated servers share identical SSH fingerprint — one with open email ports (143/993/995), consistent with phishing infrastructure.

Hostwinds ASN Pattern

C2 IP in 142.11.192.0/18 — same /18 subnet contains 3+ confirmed Lazarus IPs. Hostwinds is TA444's most-used provider (crypto payments, minimal KYC).

PoisonChain Pipeline Architecture

erything at once with `./`

Each stage feeds structured JSON into the next, enabling parallel execution where dependencies allow. The final output is a batch-generated Markdown incident report organized by team, repo, and risk level — ready for executive distribution.

Prevention & Getting Started with PoisonChain

PoisonChain GitHub Reposito

Immediate Actions

`npm ci --ignore-scripts` in CI environments

Pin `axios` to `1.14.0` with `--save-exact`

Blackhole C2: `sfrclak.com`, `callnrwise.com`

Set a release-age gate (Renovate `minimumReleaseAge` or registry-side age gate)

Alert on packages without SLSA provenance

Commit lockfiles and require diff review on changes

PoisonChain Quick Start

```
git clone https://github.com/windshock/PoisonChain
```

`cp public/.env.example .env` and fill in API keys

```
./scripts/run_full_pipeline.sh
```

Requires Python 3.9+. Built for Bitbucket/Jenkins, but you can adapt the integration points to your stack. A standalone `jenkins-scan-kit.zip` can be shared with dev teams and runs with zero external dependencies.

The value is the methodology and pipeline logic — not specific API calls. Adapt to your environment.

Long-Term Hardening Strategy

Beyond immediate response, implement these strategic controls:

GuardDog Integration

Detect postinstall scripts and obfuscated code before they execute. Phase in from monitoring → soft block → hard block. See public/docs/GUARDDOG-JENKINS-GUIDE.md.

Shared Library Policy

Centralize GuardDog policy in a Jenkins Shared Library so one change propagates to all instances.

OIDC Publishing

Migrate from personal npm tokens to OIDC-based CI/CD publishing to remove the single point of failure.

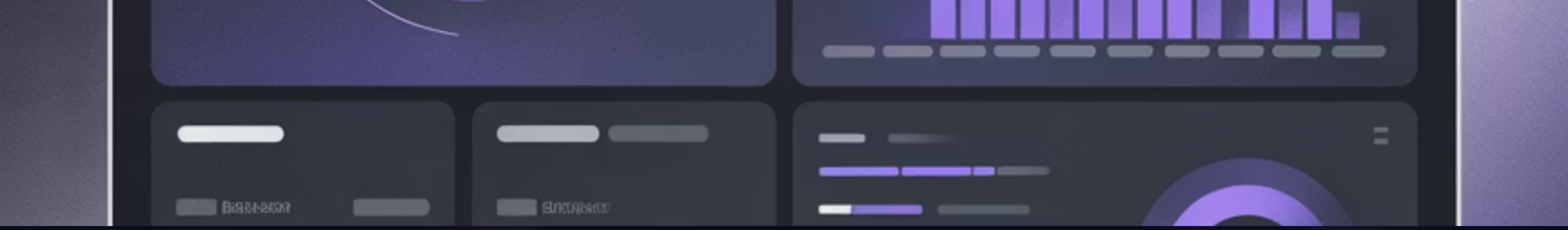
Lockfile Enforcement

Commit package-lock.json or yarn.lock and require diff review on all changes.

Minimum Release Age

Use minimumReleaseAge or a registry-side age gate to delay package adoption and create a review window.

Priority timeline: Implement GuardDog within **2 weeks**. Complete OIDC migration within **60 days**.



Jenkins Scan Kit: Standalone Forensic Toolkit

The `jenkins-scan-kit.zip` is a standalone Jenkins-specific forensic tool for scanning build history with zero external dependencies. It is designed for development teams to independently assess Jenkins instances for signs of compromise related to the malicious axios attack.

Key Features

- Runs locally with zero external dependencies.
- Scans Jenkins build history for compromise indicators.
- Works in isolated or air-gapped environments.
- Produces a `reports/jenkins-scan-result.json` output with a `risk_level` field.

How to Use

1. Extract the `jenkins-scan-kit.zip` to a local directory.
2. Copy `.env.example` to `.env` and configure your Jenkins URL and token.
3. Run `python3 jenkins_scan.py`.
4. Review results in `reports/jenkins-scan-result.json`.

The toolkit is intended specifically for Jenkins forensic analysis rather than general vulnerability scanning. It helps teams review Jenkins build history and identify suspicious activity without relying on external services or integrations.

Example Usage

```
python3 jenkins_scan.py
```

Preventive Controls: Hardening Jenkins Against Supply Chain Attacks

Proactive measures are essential to safeguard CI/CD pipelines from sophisticated supply chain attacks like PoisonChain. Implementing these controls helps detect, prevent, and mitigate risks before they impact your production environment.

GuardDog Integration

Integrate GuardDog into CI pipelines to automatically detect malicious `postinstall` scripts and obfuscated code. Implement a phased approach: monitoring, then soft-blocking, and finally hard-blocking execution.

Enforce npm ci

Always use `npm ci` instead of `npm install` for reproducible builds. Enforce the `--ignore-scripts` flag in all CI pipelines to prevent arbitrary script execution during dependency installation. `npm ci --ignore-scripts`

Shared Library Policy

Centralize GuardDog and other security policies within a Jenkins Shared Library. This ensures consistent application of rules across all build jobs and simplifies updates and enforcement across your organization.

OIDC Publishing

Migrate from personal npm tokens to OpenID Connect (OIDC) for CI/CD publishing. This eliminates static credentials and the single point of failure that personal tokens represent, enhancing overall security.

Lockfile Enforcement

Commit `package-lock.json` or `yarn.lock` to version control and enforce diff reviews on any changes. This prevents unauthorized dependency alterations and ensures consistent build environments.

npm audit Integration

Use `npm audit` as a general hygiene check, not as a primary defense against this zero-day attack. It relies on known vulnerability databases, so it would not have detected PoisonChain, and package.json swapping makes its results unreliable anyway. `npm audit --production --json`

Minimum Release Age

Set a release-age gate using Renovate `minimumReleaseAge` or a registry-side age gate to delay adoption of newly published packages. This provides a small window for malicious packages to be identified and taken down before they enter your builds.